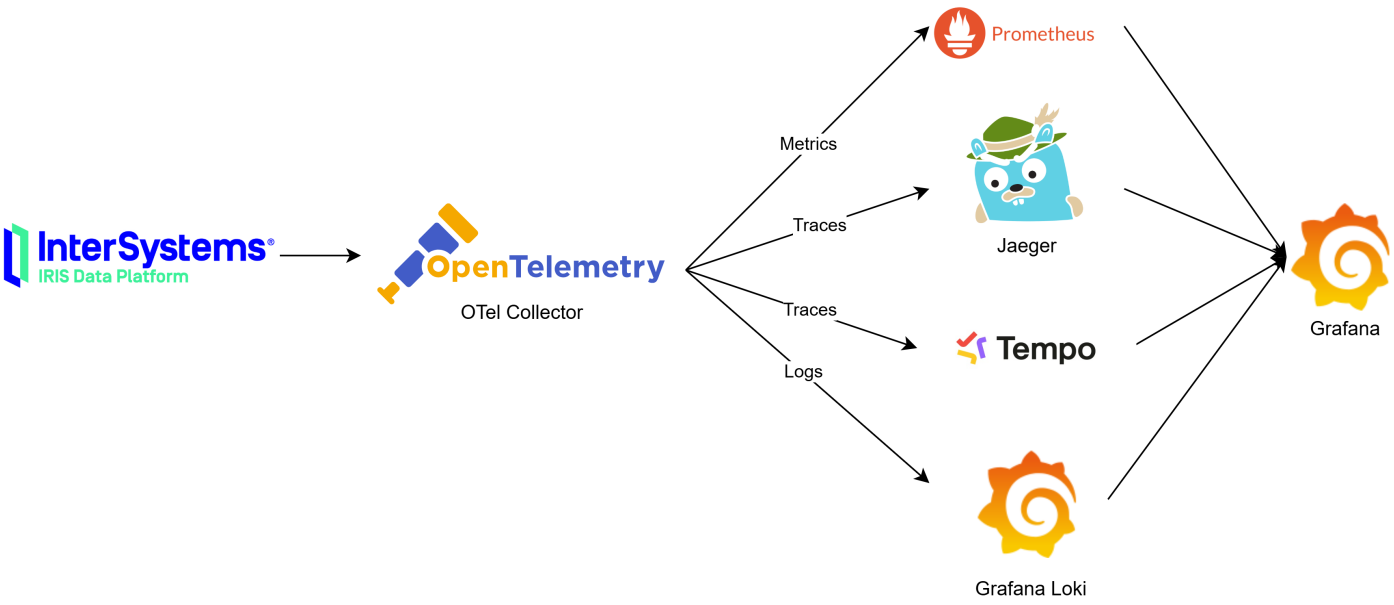
OpenTelemetry for InterSystems IRIS

This project sets up a **local OpenTelemetry environment** for **InterSystems IRIS**, providing **metrics**, **traces**, **logs** collection.
It uses Docker Compose to orchestrate all components and demonstrate how IRIS integrates with modern observability tools.



Prerequisites

- **Docker**
- **Docker Compose**

Verify installation:

```
docker --version
docker compose version
```

Quick Start

1.Start the environment

```
docker compose up -d
```

This launches:

- IRIS with a CPF merge file enabling OpenTelemetry metrics
- OpenTelemetry Collector (contrib build)
- Prometheus (with `--web.enable-remote-write-receiver`, native histograms, and exemplar storage enabled so Tempo's metrics-generator can push service-graph / span-metrics)
- Tempo (with metrics-generator + local-blocks processor)
- Jaeger
- Loki

- Grafana (datasources, dashboard, and alert rules pre-provisioned — see below)

2.How to enable OTel metrics/logs in IRIS

The **CPF merge file** is already included and automatically loaded in `docker-compose.yaml`.
It contains the following configuration:

```
[Monitor]
OTELMetrics=1
OTELLogs=1
OTELLogLevel=INFO
```

This enables the IRIS monitor to expose metrics and logs via the OpenTelemetry exporter.

3.Trigger test traces from IRIS

Open a terminal session in the IRIS container:

```
docker compose exec iris iris session IRIS
```

Then run the built-in trace demo:

```
Do ##class(%Trace.Tracer).Test()
```

This sends trace spans from IRIS to the OpenTelemetry Collector, which then exports them to Jaeger.

---

4. (Optional) Run a second IRIS instance

A second IRIS service (`iris-2`) is defined under a Compose profile so it stays off by default. Start it when you want to demo multi-host monitoring:

```
docker compose --profile multi-iris up -d
```

It registers with the collector as `OTEL_SERVICE_NAME=iris-2`, so it appears as a separate entry in the `IRIS service` dropdown on the **IRIS Overview** dashboard and in alert labels.

Trigger traces from it the same way:

```
docker compose exec iris-2 iris session IRIS
```

```
Do ##class(%Trace.Tracer).Test()
```

Stop just the extra instance with `docker compose stop iris-2`.

Viewing Results

Jaeger - Traces

Open http://localhost:16686

1. Select your **Service Name** ("test_service" in this example)
2. Click **Find Traces**
3. Expand a trace to view span details, durations, and relationships

---

Prometheus - Metrics

Open http://localhost:9090

Try example queries:

```
iris_db_size_mb_megabytes
iris_cpu_usage
iris_res_seize_total
```

Grafana - Traces + Metrics + Logs

Open http://localhost:3000 (anonymous admin is enabled).

Pre-provisioned datasources: Prometheus (default), Loki, Tempo, Jaeger with exemplar links from Prometheus -> Tempo, derived `trace_id=` field from Loki -> Tempo, and Tempo service map + trace-to-logs/metrics wired up.

Pre-provisioned dashboard: `IRIS Overview` (folder `IRIS`) -> http://localhost:3000/d/iris-overview

Panels: CPU usage, database size, resource-seize rate, per-instance time series, Tempo service graph, and live IRIS logs. Use the `Instance` dropdown to filter.

Drilldown view: http://localhost:3000/drilldown

Pre-provisioned alerts (folder `IRIS`, http://localhost:3000/alerting/list):

Rule	Condition
IRIS CPU usage high	`avg by (instance) (iris_cpu_usage) > 80` for 2m
IRIS DB growth high	`sum(increase(iris_db_size_mb_megabytes[10m])) > 100` for 5m

To route alerts somewhere, add a contact point + notification policy under `configs/grafana/provisioning/alerting/`.

Debug output

You can also view logs and traces from debug output

```
docker compose logs -f otel-collector
```

Cleanup

```
docker compose down -v
```